



Препроцессоры CSS

Препроцессоры CSS

Sass. Появился в 2006 году как способ упростить работу с CSS и сразу стал популярен у разработчиков. Некоторые считают, что у Sass невысокая скорость работы, но на практике это можно заметить только на очень больших проектах.

- **Sass** - самый первый вариант, где не используются фигурные скобки, а всё сделано на отступах, как в Python.
- **SCSS** - в нём есть фигурные скобки, как в CSS, а ещё есть поддержка современных CSS-трюков, которые используются в разных браузерах. Этот формат появился после выхода LESS и стал ответом конкуренту.

LESS. В 2009 году Alexis Sellier показал свою версию препроцессора, основанную на Sass. Он взял за основу чистый CSS-синтаксис и добавили в него возможность использовать все расширения как в Sass.

Stylus. Был разработан в 2011 году. Принципиально он делает то же самое, но использует другой синтаксис, более лаконичный и простой.

Препроцессоры CSS

Причины использования CSS-препроцессоров

Возможность записать код короче

В стандартном CSS нет возможности делать вложенности, создавать переменные. Это сильно усложняет написание большого объема стилей, создает необходимость дублировать одинаковые участки кода, а также помнить название всех ID и классов. Благодаря препроцессорам разработчик:

- не должен запоминать множество классов, следовательно, вероятность допустить ошибку в названии, из-за которой слетит половина стилей, ниже;
- получает возможность структурировать код по вложениям, что делает удобным его зрительное восприятие и ускоряет процесс разработки;
- имеет возможность задавать переменные и функции, сокращает время на написании повторяющихся стилей и уменьшает вероятность ошибок;
- тратит меньше времени на процесс прописывания стилей.

Препроцессоры CSS

Причины использования CSS-препроцессоров

Легкость изучения

Благодаря тому, что синтаксис препроцессоров слабо отличается от синтаксиса CSS, разработчик, освоивший стандартный CSS может почти сразу же начать делать проекты с применением препроцессоров.

Простота применения

С каждым новыми версиями CSS-препроцессоры становится все проще встроить в проект. Если раньше требовалось скачать несколько программ, плагинов, выполнить их настройку через терминал, то сейчас разработчик может воспользоваться лишь одной программой-компилятором. Сейчас такие программы предоставляют почти все необходимое, что избавляет от практической необходимости долго «возиться» с настройкой проекта.

Препроцессоры CSS

Причины использования CSS-препроцессоров

Логичная и понятная структура

Возможность создавать вложения является главным преимуществом любого CSS-препроцессора. Такой код выглядит логичным, его удобно читать и редактировать, плюс, гораздо меньше риск допустить ошибку.

Препроцессор CSS Stylus

Stylus – это препроцессор CSS, написанный на языке JavaScript для Node.js. После интерпретации сервером синтаксис Stylus превращается в CSS код.

Например, добавим стили для страницы «Hello, world!» (файл **index.pug** проекта):

```
html(lang="ru")
  head
    meta(charset="utf-8")
    title= "Пример Pug"
    link(rel="stylesheet", href="css/main.css")
  body
    h1 Hello, World!
```

В результате генерируется страница **index.html**, к которой применяются стили, описанный в файле **main.styl**.

Синтаксис Stylus

Синтаксис Stylus основан на следующих положениях:

- обязательная вложенность свойств обозначается либо пробелами, либо табуляцией;
- нет фигурных скобок;
- нет точек с запятой;
- нет двоеточий (но можно и ставить).

Синтаксис Stylus

Код CSS

```
body {  
  font-size: 12px;  
  color: Navy;  
}  
p.text {  
  padding: 5px;  
  margin: 5px;  
  border: solid thin Navy;  
  border-radius: 5px;  
}
```

Код Stylus

```
body  
  font-size 12px  
  color Navy  
  
p.text  
  padding 5px  
  margin 5px  
  border solid thin Navy  
  border-radius 5px
```

Переменные

В коде Stylus можно использовать переменные, которым можно задавать значения, используя оператор присваивания.

Имена переменных определяются в соответствии с обычными правилами, могут включать знак \$, их можно использовать в правых частях операторов присваивания других переменных.

Переменные

Код Stylus

```
gap = 5px
col = Navy

body
  font-size 12px
  color col

p.text
  padding gap
  margin gap
  border solid thin col
  border-radius gap
```

Код CSS

```
body {
  font-size: 12px;
  color: Navy;
}

p.text {
  padding: 5px;
  margin: 5px;
  border: solid thin Navy;
  border-radius: 5px;
}
```

Переменные

Код Stylus

```
gap = 5px
col = Navy
$bord = solid thin col
```

```
body
  font-size 12px
  color col

p.text
  padding gap
  margin gap
  border $bord
  border-radius gap
```

Код CSS

```
body {
  font-size: 12px;
  color: Navy;
}

p.text {
  padding: 5px;
  margin: 5px;
  border: solid thin Navy;
  border-radius: 5px;
}
```

Значения свойств

При описании стилей можно использовать значения уже заданных свойств, при этом перед названием этого свойства при определении стиля необходимо добавить @.

Код Stylus

```
p.text
  padding 5px
  margin @padding
  border solid thin Navy
  border-radius @margin
```

Код CSS

```
p.text {
  padding: 5px;
  margin: 5px;
  border: solid thin Navy;
  border-radius: 5px;
}
```

Переменные

Задание 1. Как будет на Stylus выглядеть следующий CSS код (все числовые значения, кроме 0, должны быть выражены через переменную **size**).

Код CSS

```
section {  
  box-shadow: 1% 1% 2% rgba(0,0,0,0.5);  
  padding: 2%;  
  margin: 4%;  
}
```

Соединение селекторов

Если несколько селекторов имеют одинаковый стиль их можно перечислить через запятую или записать на следующей строке (все селекторы с одинаковым отступом).

Код Stylus

```
h1, h2  
  color navy  
  
или  
  
h1  
h2  
  color navy
```

Код CSS

```
h1,  
h2 {  
  color: navy;  
}
```

Родительские селекторы

Символ **&** ссылается на родительский селектор(ы). Дочерние селекторы должны быть указаны с отступом относительно родительского селектора.

Пример. Изменим цвет заголовков при наведении на них курсора мыши с помощью псевдоселектора **:hover**.

Код Stylus

```
h1
h2
  color navy
  &:hover
    color red
```

Код CSS

```
h1,
h2 {
  color: navy;
}
h1:hover,
h2:hover {
  color: red;
}
```

Родительские селекторы

Пример. Опишем класс **menu**, являющийся потомком класса **list**. И для класса **list** зададим псевдоэлемент.

Код Stylus

```
.list
  & .menu
    color: navy

  &:hover
    color: red
```

Код CSS

```
.list .menu {
  color: #000080;
}
.list:hover {
  color: #ff0000;
}
```


Родительские селекторы

Пример. Опишем класс **menu**, являющийся потомком класса **list**. И зададим псевдоэлемент для этого класса.

Код Stylus

```
.list
  & .menu
    color: navy

  &:hover
    color: red
```

Код CSS

```
.list .menu {
  color: #000080;
}
.list .menu:hover {
  color: #ff0000;
}
```

Родительские селекторы

Пример. Опишем класс **menu**, являющийся элементом блока **list** (методология БЭМ) и укажем модификатор для этого элемента.

Код Stylus

```
.list
  &__menu
    color: navy

  &__active
    color: red
```

Код CSS

```
.list__menu {
  color: #000080;
}
.list__menu_active {
  color: #ff0000;
}
```

Родительские селекторы

Задание 2. Написать код Stylus, чтобы после трансляции HTML-код выглядел следующим образом:

```
.block {  
  display: flex;  
  flex-wrap: wrap;  
}  
.block p {  
  display: inline-block;  
  background: grey;  
}  
.block p:hover {  
  color: green;  
}  
  
.block:hover {  
  color: red;  
}
```

Частичная ссылка

Частичная ссылка работает аналогично родительской ссылке, но родительская ссылка содержит весь селектор, а частичная ссылка позволяет обратиться к любому уровню вложенных селекторов.

Записывается частичная ссылка так:

^[N]

где **N** – номер уровня вложенности.

Например:

^[0] выделяет селектор первого уровня;

^[1] выделяет селектор второго уровня и так далее.

Частичная ссылка

Код Stylus

```
.list
  &__menu
    color: navy

  ^[0]:hover &
    color: red;
```

Код CSS

```
.list__menu {
  color: #000080;
}
.list:hover .list__menu {
  color: #ff0000;
}
```

Частичная ссылка указывает, что псевдоэлемент применяется к селектору первого уровня, то есть к классу `.list`.

Частичная ссылка

Код Stylus

```
.list
  &__menu
    color: navy

  &_active
    color: red

  ^[-1]:hover &
    color: blue;
```

Код CSS

```
.list__menu {
  color: #000080;
}
.list__menu_active {
  color: #ff0000;
}
.list__menu:hover .list__menu_active {
  color: #0000ff;
}
```

Отрицательные значения считаются с конца `^[-1]` определяет последний селектор в цепочке перед `&`.

Частичная ссылка

Задание 3. Как будет выглядеть CSS-код после преобразования кода Stylus

```
.block
  color blue
& .menu
  color navy
& a
  color red
& h1
  color green
  ^[2]:hover
    color: white;
```

Миксины

Если какие-то части кода используются несколько раз, их можно выделить отдельно.

Миксины могут иметь параметры.

Миксины

Код Stylus

```
body
  font-size 12px
  color Navy
// описание миксина
style-p(size)
  padding size
  margin size
  border-radius size

p.text
  style-p(5px) // вызов миксина
  border solid thin Navy
```

Код CSS

```
body {
  font-size: 12px;
  color: Navy;
}
p.text {
  padding: 5px;
  margin: 5px;
  border: solid thin Navy;
  border-radius: 5px;
}
```

Миксины

Формальный параметр в скобках можно не указывать.

Код Stylus

```
body
  font-size 12px
  color Navy
// описание миксина
style-p()
  padding arguments
  margin arguments
  border-radius arguments

p.text
  style-p(5px) // вызов миксина
  border solid thin Navy
```

Код CSS

```
body {
  font-size: 12px;
  color: Navy;
}
p.text {
  padding: 5px;
  margin: 5px;
  border: solid thin Navy;
  border-radius: 5px;
}
```

Миксины

В миксинах можно использовать неопределенное количество параметров, миксины можно вызывать из других миксинов.

Код Stylus

```
body
  font-size 12px
  color Navy

style-border(args...)
  border args

style-p(size, col)
  padding size
  margin size
  border-radius size
  style-border(solid, thin, col)

p.text
  style-p(5px, Navy)
```

Код CSS

```
body {
  font-size: 12px;
  color: Navy;
}

p.text {
  padding: 5px;
  margin: 5px;
  border: solid thin Navy;
  border-radius: 5px;
}
```

Миксины

В миксинах в качестве параметров можно передавать название свойств или их часть, в коде миксина они заключаются в фигурные скобки:

Код Stylus

```
body
  font-size 12px
  color Navy

style-p(part, size, col)
  padding size
  margin size
  border-{part} size
  border solid thin col

p.text
  style-p(radius, 5px, Navy)
```

Код CSS

```
body {
  font-size: 12px;
  color: Navy;
}

p.text {
  padding: 5px;
  margin: 5px;
  border: solid thin Navy;
  border-radius: 5px;
}
```

Миксины

Задание 4. Создать миксин `style-tag()`, который в качестве параметра получает название тега и цвет. И устанавливает этот цвет всем элементам с указанным тегом, являющимися дочерними элементами некоторого другого тега.

Код Stylus:

```
div
  style-tag(ul, Navy)
```

Код HTML

```
div ul {
  color: Navy;
}
```

Код Stylus:

```
article
  style-tag(p, Red)
```

Код HTML

```
article p {
  color: Red;
}
```

Вычисления

Stylus допускает вычисления в правых частях операторов присваивания и при описании свойств стилей.

Арифметические операции

Stylus поддерживает стандартные арифметические операции: сложение (+), вычитание (-), умножение (*), деление (/), остаток от деления (%).

При выполнении операций происходит автоматическое преобразование абсолютных единиц измерения (**cm**, **mm**, **in**, **s**, **ms**). В остальных случаях вычисления осуществляется только с числами, если таковые можно выделить.

15px - 5px => 10px

5 - 2 => 3

5in - 50mm => 3.031in

20mm + 4in => 121.6mm

15pt - 5px => 10pt

"auto " + 15px => "auto 15px"

Вычисления

Пример. Элемент класса **main** имеет ширину 700 пикселей. Разместить в этом блоке три одинаковых рисунка с отступом, так чтобы они занимали всю ширину блока.

Код Stylus

```
.main
  width: 700px
  &__img
    margin: 10px
    width: ((@width - 2 * @margin) / 3)
```

Код CSS

```
.main {
  width: 700px;
}
.main__img {
  margin: 10px;
  width: 226.66666666666666px;
}
```

Математические функции

При вычислениях можно использовать математические функции типа **floor()**, **round()**, **ceil()**, **max()**, **min()**, **abs()**, **sum()**, **avg()** и пр.

sum(2 4 1 7) => 14

avg(2px 4 1px 7) => 3.5px

round(4.8456px, 2) => 4.85px

max(1px, 7) => 7

Вычисления

Пример. Элемент класса **main** имеет ширину 700 пикселей. Разместить в этом блоке три одинаковых рисунка с отступом, так чтобы они занимали всю ширину блока.

Код Stylus

```
.main
  width: 700px
  &__img
    margin: 10px
    width: floor((@width - 2 * @margin) / 3)
```

Код CSS

```
.main {
  width: 700px;
}
.main__img {
  margin: 10px;
  width: 226px;
}
```


Вычисления

Задание. Реализовать миксин, который в качестве параметра получает 3 целых числа в интервале от 0 до 255, обозначающие долю красного, зеленого и синего цвета соответственно. В миксине должен формировать «инверсионный» цвет на основе переданных параметров, а результат присваиваться свойству **color**.

Код Stylus:

```
div
  color-inv(0, 0, 0)
```

Код HTML

```
div {
  color: #fff;
}
```

Код Stylus:

```
div
  color-inv(255, 0, 0)
```

Код HTML

```
div {
  color: #0ff;
}
```

Функции

В Stylus можно создавать функции с параметрами, которые вычисляют и возвращают значение одного выражения:

имя(параметр1, параметр2)
выражение

Можно установить значение параметра по умолчанию:

имя(параметр1, параметр2=значение)
выражение

Функции

Пример. Создадим функцию, которая вычисляет ширину рисунка в зависимости от ширины блока и количества размещаемых в нем рисунков.

Код Stylus

```
getWidth(w, mar, n=3)
  floor((w - 2 * mar) / n)

.main
  width: 700px
  &__img
    margin: 10px
    width: getWidth(@width, @margin, 4)
```

Код CSS

```
.main {
  width: 700px;
}
.main__img {
  margin: 10px;
  width: 170px;
}
```

Функции

Если же при вызове пропустить последний параметр, то будет выполнен расчет на 3 картинки.

Код Stylus

```
getWidth(w, mar, n=3)
  floor((w - 2 * mar) / n)

.main
  width: 700px
  &__img
    margin: 10px
    width: getWidth(@width, @margin)
```

Код CSS

```
.main {
  width: 700px;
}
.main__img {
  margin: 10px;
  width: 226px;
}
```

Условный оператор

В коде Stylus можно использовать условный оператор. Его синтаксис:

```
if условие
  оператор Stylus
else
  оператор Stylus
```

Условие – это логическое выражение, в котором можно использовать:

- знаки отношения (`==` или **is**, `!=` или **is not**, `>=`, `<=`, `>`, `<`);
- логические операции И (**and** или `&&`), ИЛИ (**or** или `||`), НЕ (**not** или `!`);
- оператор **in**;

Условный оператор

Примеры логических выражений:

```
10 < 5px => false
#fff == #fff => true
#fff == #ffffff => true
#fff is #ffffff => true
(1 2 3) == (1 2 3) => true
values = 1 2 3
1 in values => true
5 in values => false
5 and 3 => 3
5 || 3 => 5
1 in values or 5 in values => true
```

Условный оператор

Пример. Проверим, есть ли в описании внешнего отступа элемента значение auto, если нет, то добавим выравнивание по центру.

Код Stylus

```
div
  margin 0 auto
  if not ("auto" in @margin)
    text-align center
```

Код CSS

```
div {
  margin: 0 auto;
}
```

Условный оператор

Пример. Проверим, есть ли в описании внешнего отступа элемента значение auto, если нет, то добавим выравнивание по центру.

Код Stylus

```
div
  margin 0 10
  if not ("auto" in @margin)
    text-align center
```

Код CSS

```
div {
  margin: 0 auto;
  text-align: center;
}
```

Постфиксный условный оператор

Stylus поддерживает постфиксные условные операторы:

оператор **if** выражение

Оператор выполняется только в том случае, если выражение истинно.

Постфиксный условный оператор

Пример. Проверим, есть ли в описании внешнего отступа элемента значение **auto**, если нет, то добавим выравнивание по центру, в противном случае – выравнивание по правой границе.

Код Stylus

```
align(marg)
  return text-align center if not ("auto" in marg)
  text-align right
```

```
div
  margin 0 auto
  align(@margin)
p
  margin 0 10
  align(@margin)
```

Код CSS

```
div {
  margin: 0 auto;
  text-align: right;
}
p {
  margin: 0 10;
  text-align: center;
}
```

Оператор повторения

В коде Stylus можно использовать операторы повторения. Синтаксис оператора **for**:

for val_name **in** expression
операторы

При выполнении цикла управляющая переменная на каждом шаге принимает последовательные значения элементов списка, перечисленные после **in** через пробел:

Код Stylus:

```
div
  for i in 1 4 6
    .block-{i}
      width 10% * i
```

Код CSS:

```
div .block-1 {
  width: 10%;
}
div .block-4 {
  width: 40%;
}
div .block-6 {
  width: 60%;
}
```

Оператор повторения

Вместо списка можно использовать диапазон значений:

Код Stylus:

```
div
  for i in 1..3
    .block-{i}
      width 10% * i
```

Код CSS:

```
div .block-1 {
  width: 10%;
}
div .block-2 {
  width: 20%;
}
div .block-3 {
  width: 30%;
}
```


Спасибо за внимание!